



Master in Computer Science
Software Engineering

Cloud Computing

Final Report

Final Lab – UrbanMove Smart Mobility on AWS

Team Members

Jordy Meye
Huizhe Su
Arihant Jain
Nizar

April 2026

Contents

1	Objective and Scope	2
1.1	Final lab objective	2
1.2	Implemented repositories and components	2
2	Architecture and Traffic Flow	2
2.1	Architecture diagram	2
2.2	Request path	2
2.3	Outbound external feed path	3
3	Infrastructure Provisioning (AWS CLI)	3
3.1	Provisioning command sequence	3
3.2	Provisioning evidence	3
4	Service Deployment and API Flow	7
4.1	Deployment command sequence	7
4.2	Routing and access model implemented	8
5	Frontend Implementation	9
5.1	UI files deployed	9
5.2	UI flow	9
6	Observability, Cost Control, and Cleanup	10
6.1	Commands used	10
6.2	Evidence screenshots	11
6.3	Backup and restore drill (non-destructive)	12
7	Evidence Mapping	13
7.1	Requirement to evidence map	13
8	Conclusion	13

Exercise 1. Objective and Scope

Question 1. Final lab objective

Implement and deploy a 3-service mobility platform on AWS with:

- private runtime network for services,
- controlled outbound traffic through NAT,
- IAM baseline for deploy/runtime identities,
- WAF protection at CloudFront edge,
- API entry through API Gateway to private services.

Question 2. Implemented repositories and components

- `lab_final/repos/urbanmove-auth`
- `lab_final/repos/urbanmove-ingestion-routing`
- `lab_final/repos/urbanmove-api`
- `lab_final/frontend/static` (multi-page UI: home, login, operations, control)
- `lab_final/infra/aws-cli` (infra provisioning)
- `lab_final/infra/deploy` (service deployment and smoke test)

Exercise 2. Architecture and Traffic Flow

Question 1. Architecture diagram

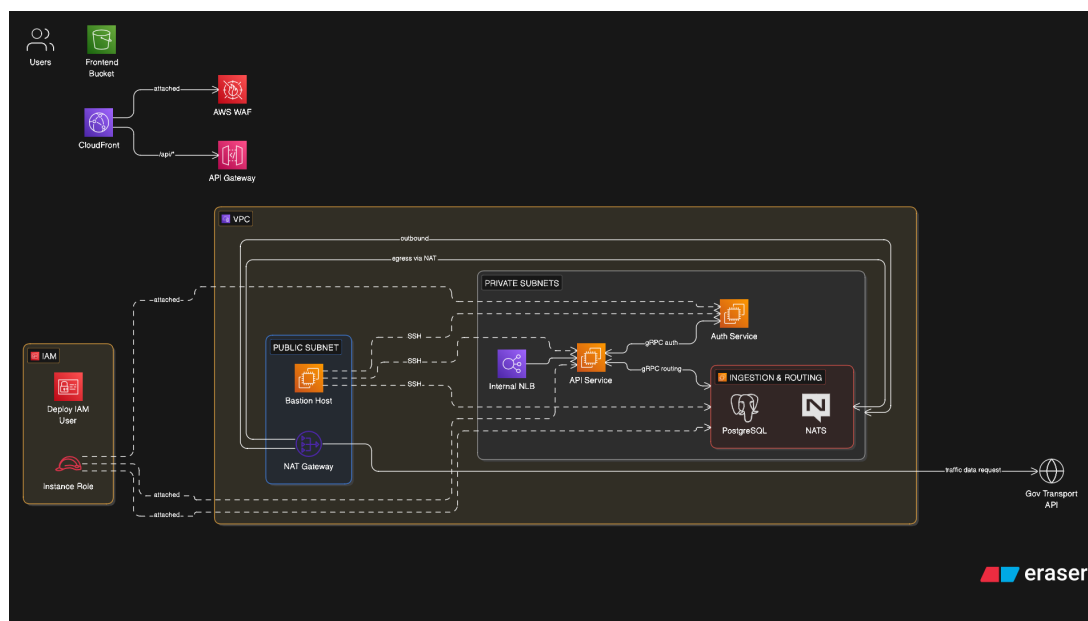


Figure 1: Final architecture exported from Eraser

Question 2. Request path

1. User requests UI/API via CloudFront.
2. CloudFront routes:
 - default path to S3 frontend bucket,

- /api/* path to API Gateway.
- API Gateway forwards private integration traffic through VPC Link to internal NLB.
 - Internal NLB forwards to API service in private subnet.
 - API service calls Auth and Ingestion/Routing over gRPC.

Question 3. Outbound external feed path

Traffic to the government transport API is initiated by private services and exits via NAT Gateway and Internet Gateway.

Exercise 3. Infrastructure Provisioning (AWS CLI)

Question 1. Provisioning command sequence

```
cd lab_final/infra/aws-cli

./01_budget_and_tags.sh
./01a_identity_iam_setup.sh
./02a_create_vpc_private_network.sh
./02_create_ec2_topology.sh
./03a_api_gateway_setup.sh
./03_s3_cloudfront_setup.sh
./04_cloudwatch_setup.sh
./06_waf_setup.sh
```

Question 2. Provisioning evidence

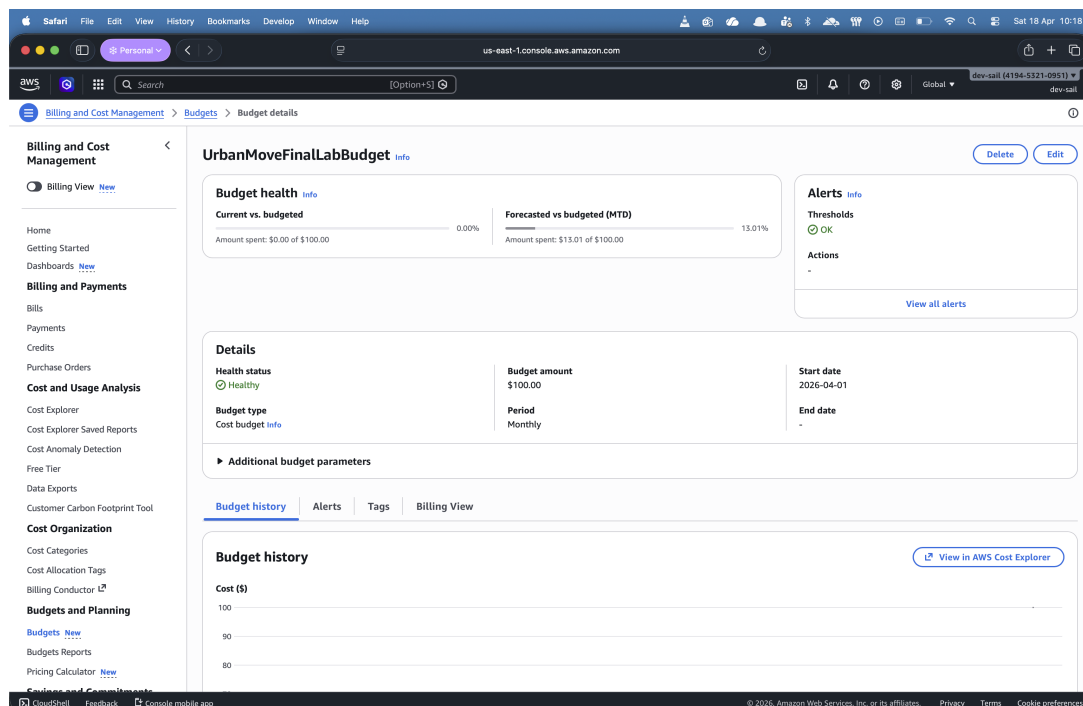


Figure 2: AWS Budget alert configuration

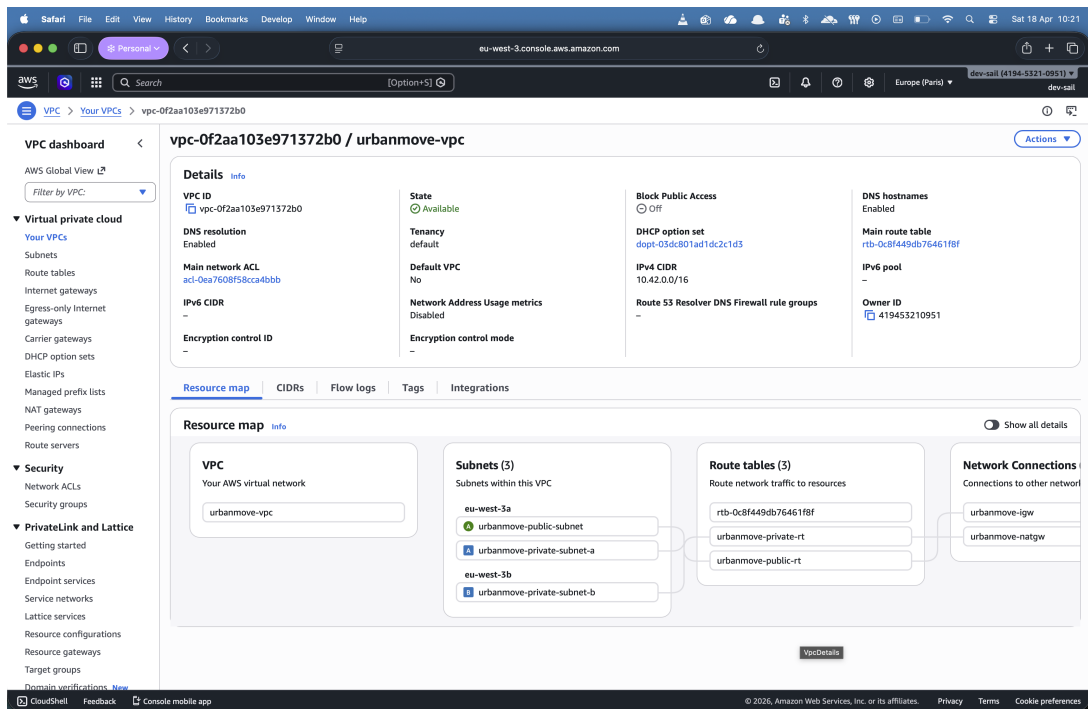


Figure 3: VPC with public/private subnets and routes

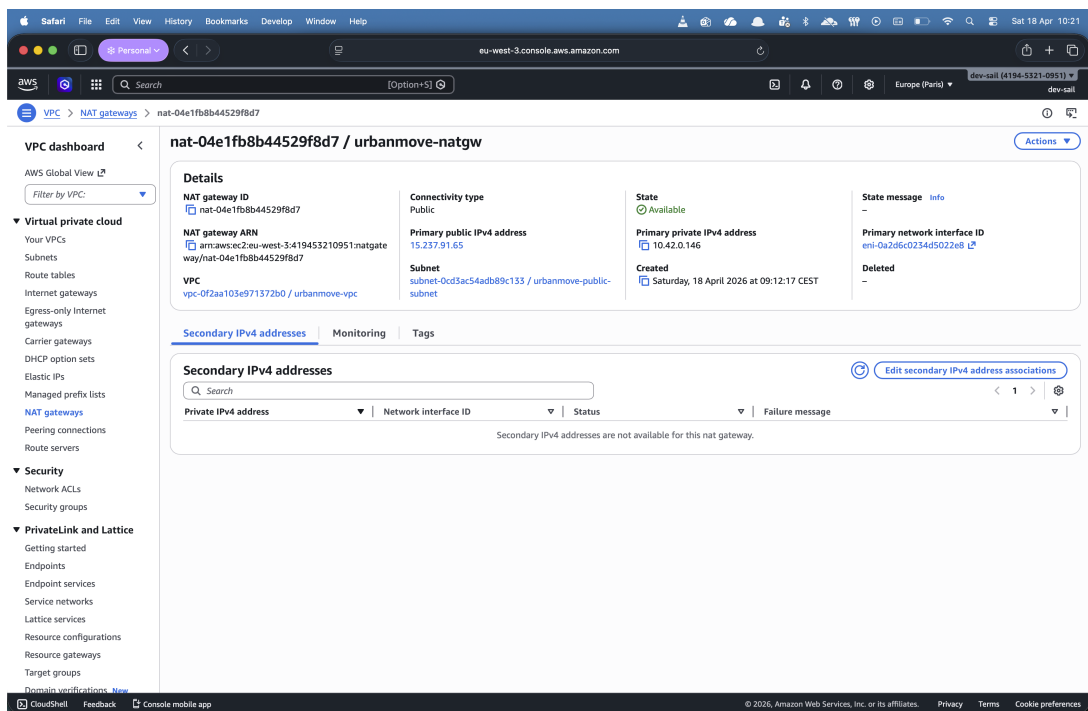


Figure 4: NAT Gateway created in public subnet

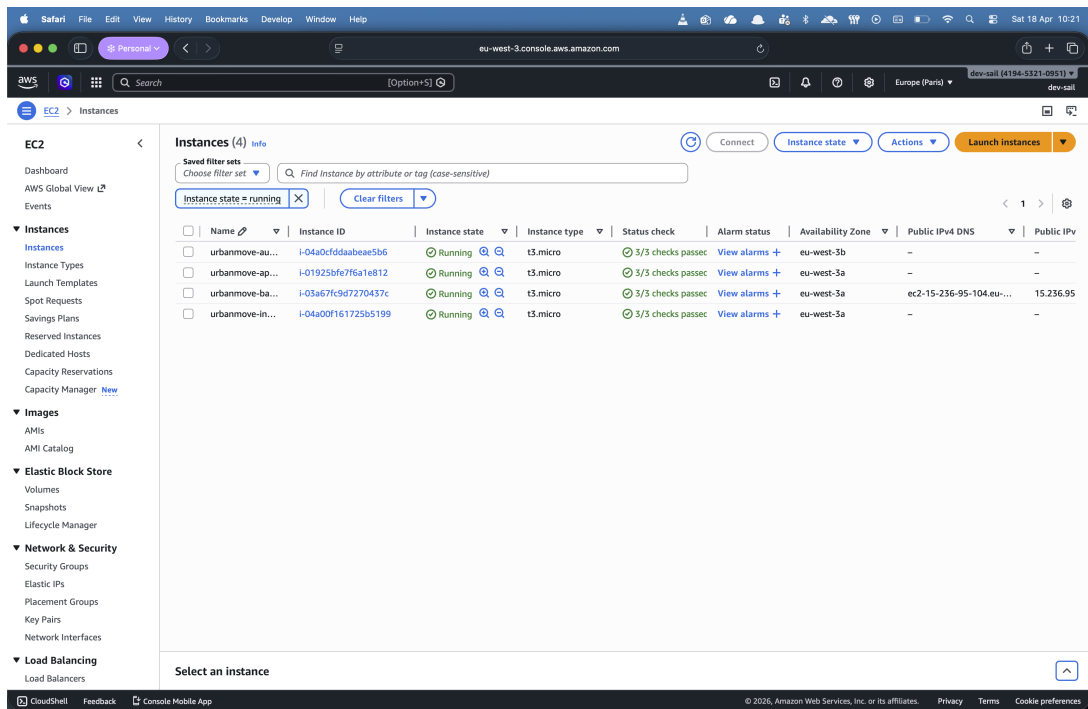


Figure 5: EC2 topology (bastion + service hosts)

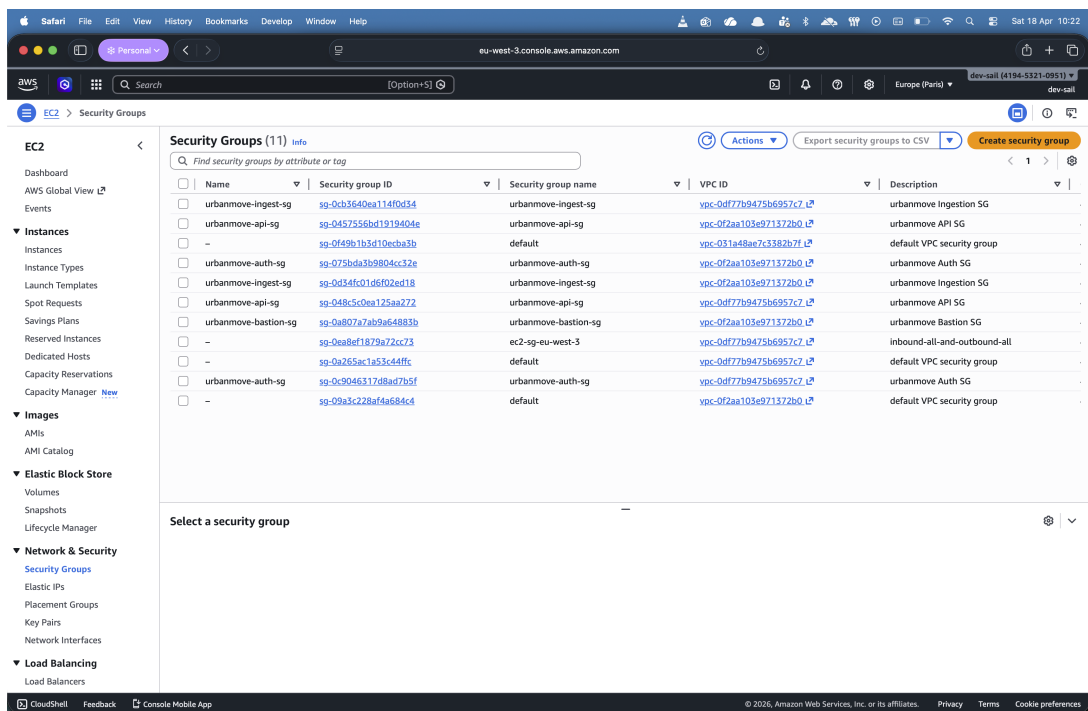


Figure 6: Security groups for bastion and services

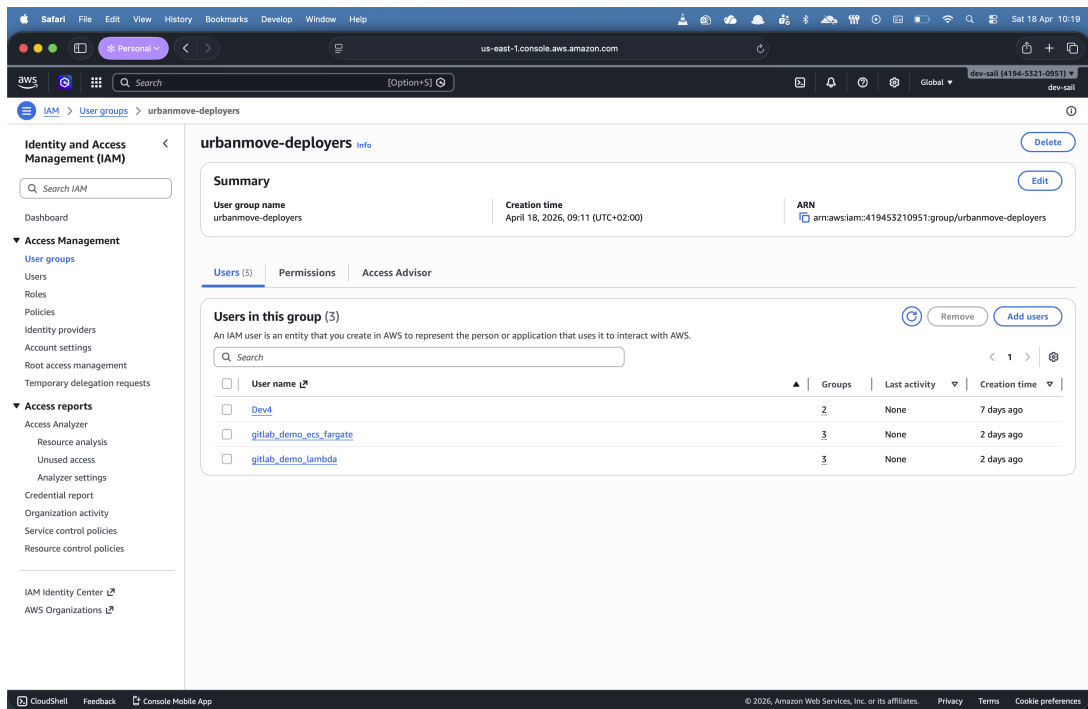


Figure 7: IAM users and identity artifacts

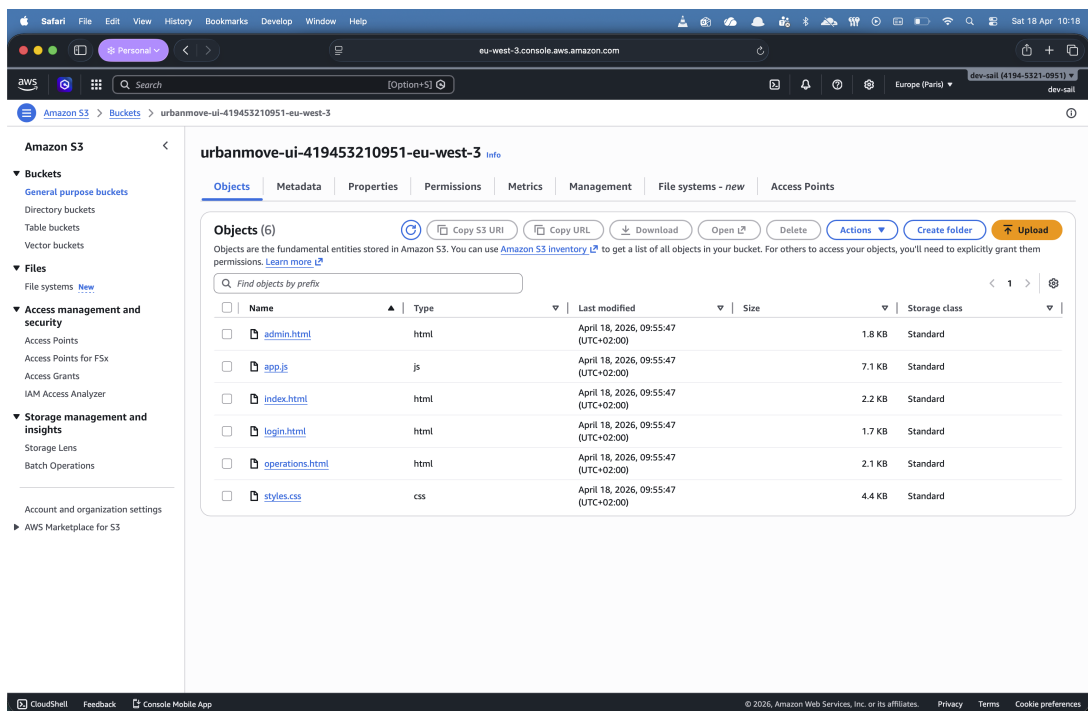


Figure 8: S3 frontend bucket

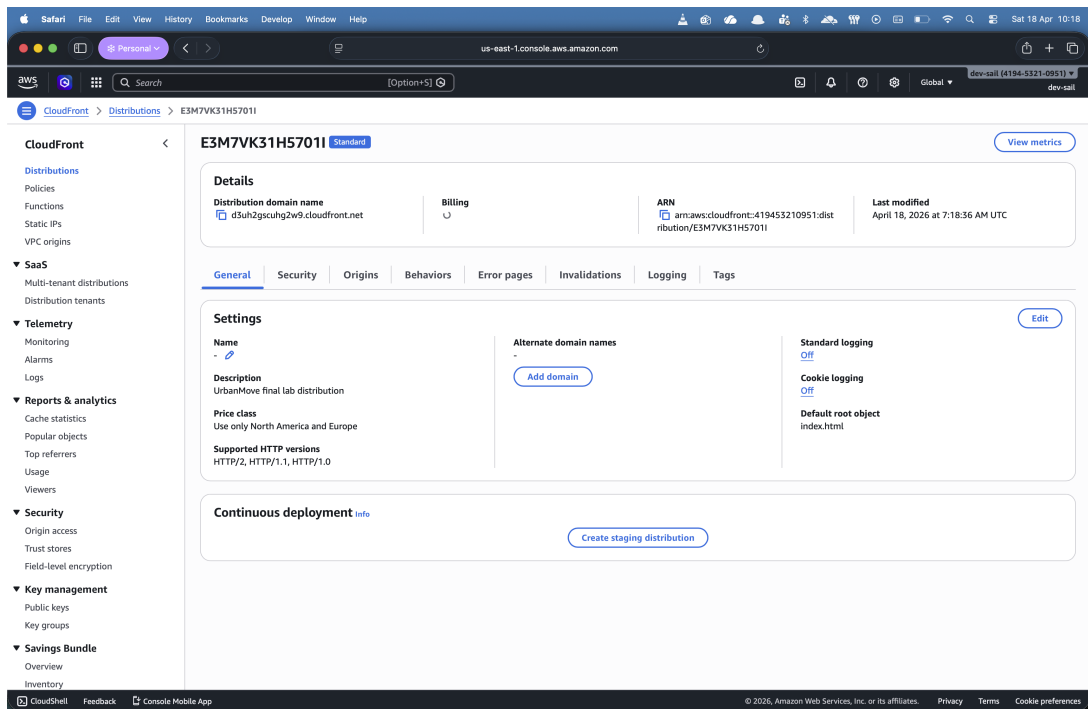


Figure 9: CloudFront distribution

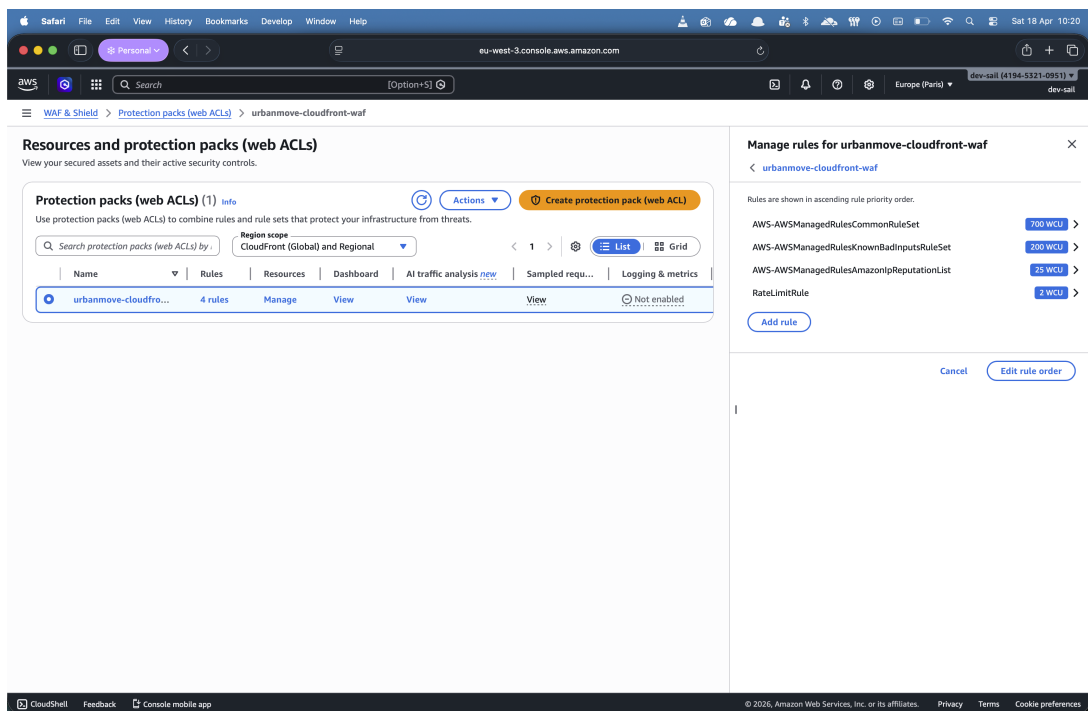


Figure 10: WAF Web ACL associated with CloudFront

Exercise 4. Service Deployment and API Flow

Question 1. Deployment command sequence

```
cd lab_final/infra/deploy

./01_bootstrap_hosts.sh
./02_deploy_auth.sh
./03_deploy_ingest.sh
./04_deploy_api.sh
./05_upload_frontend.sh
./06_smoke_test.sh
```

Question 2. Routing and access model implemented

- API routing exposed through API Gateway path `/api/*`.
- API Gateway private integration configured to internal NLB listeners.
- Deploy scripts use bastion tunnel for private hosts.
- Smoke test validates login, congestion, route recommendation, and event APIs.

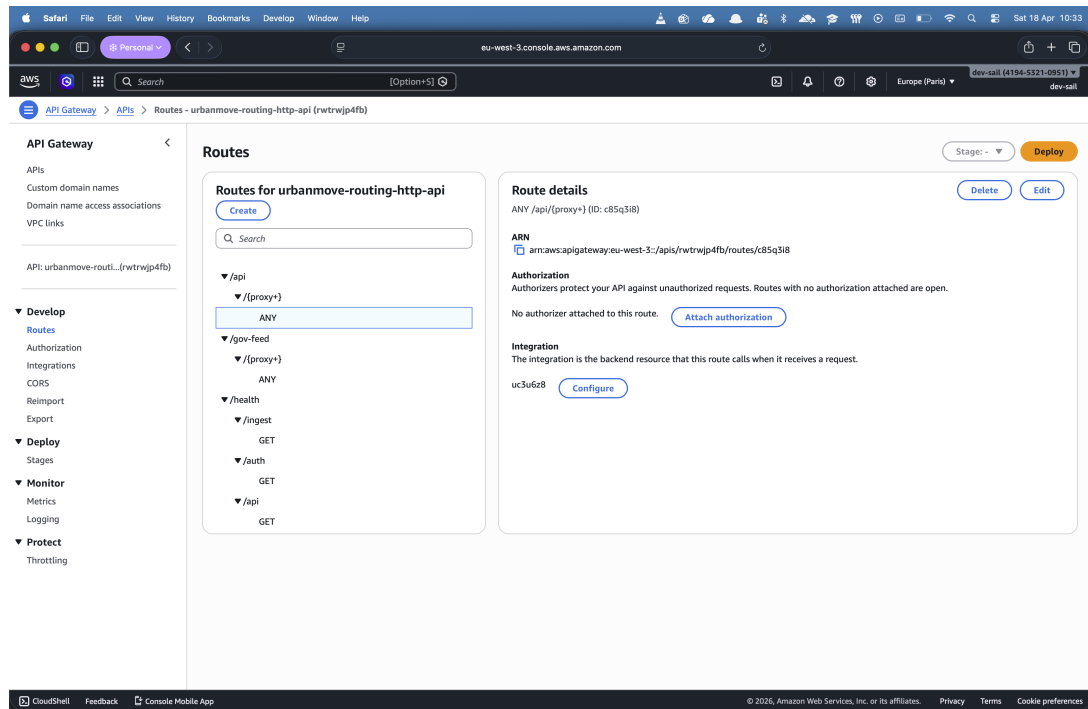


Figure 11: API Gateway route and integration view

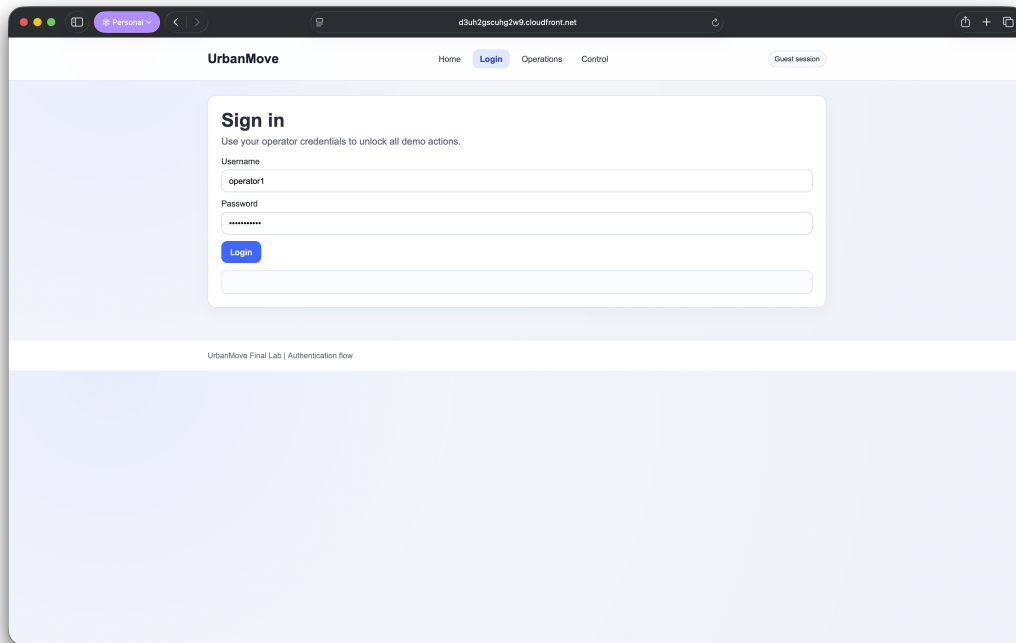


Figure 13: Frontend login page

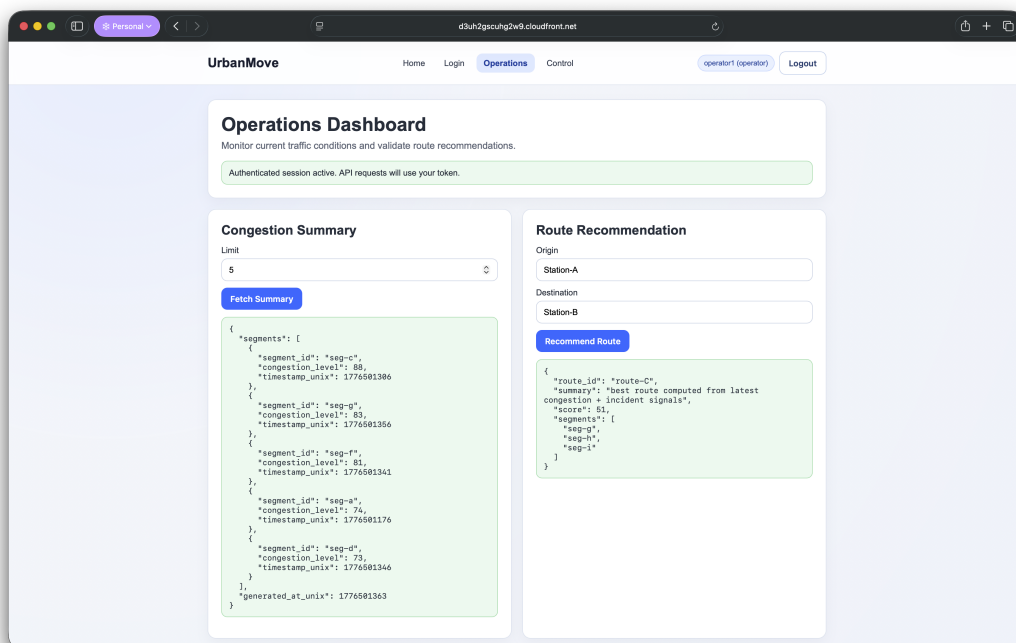


Figure 14: Frontend operations/control page

Exercise 6. Observability, Cost Control, and Cleanup

Question 1. Commands used

```
cd lab_final/infra/aws-cli

# CloudWatch dashboard and alarms
./04_cloudwatch_setup.sh

# Daily cleanup dry run
bash ./05_daily_cleanup.sh

# Apply cleanup actions
APPLY=true bash ./05_daily_cleanup.sh
```

Question 2. Evidence screenshots

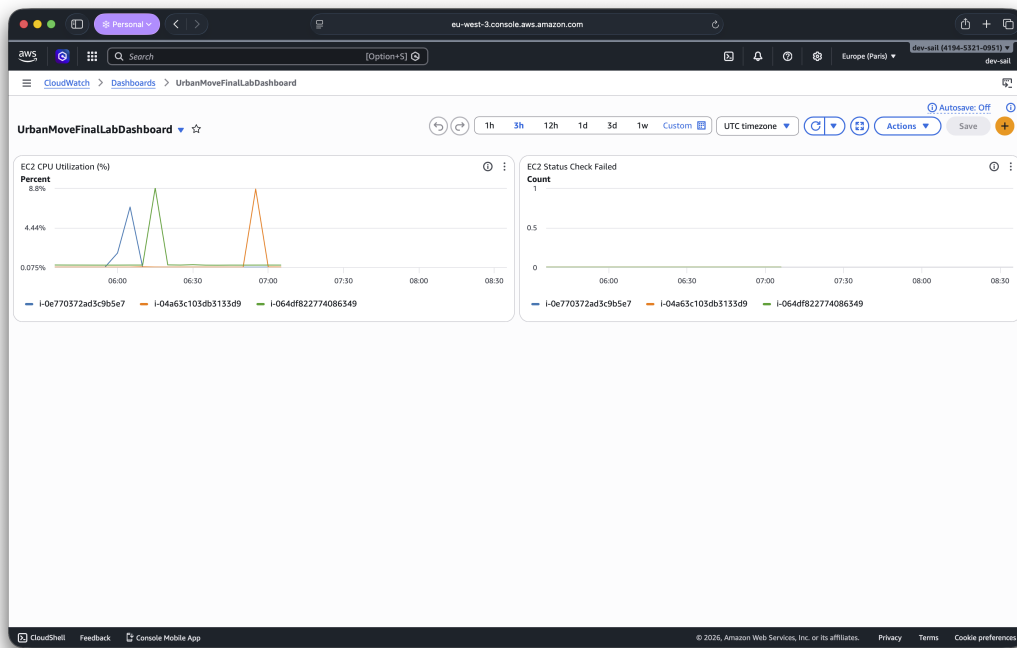


Figure 15: CloudWatch dashboard and alarm evidence

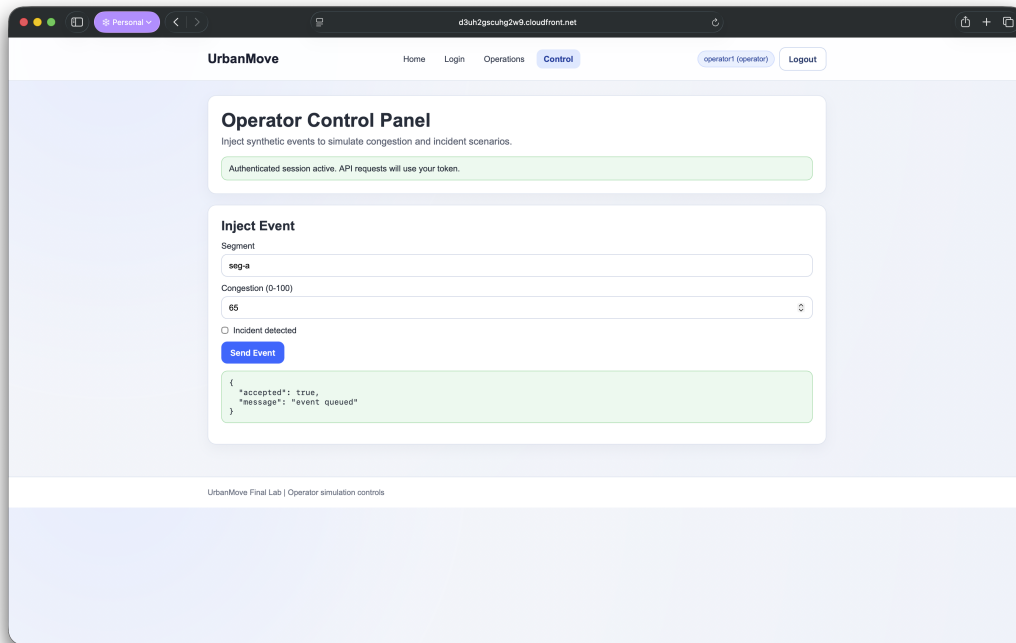


Figure 16: CI/CD pipeline run evidence for services

Question 3. Backup and restore drill (non-destructive)

```
# SSH to ingest host
cd lab_final/infra/deploy
source ./common.sh
ssh "${SSH_OPTS[@]}" "${SSH_USER}@${INGEST_HOST}"

# Backup production database
sudo docker exec urbanmove-postgres pg_dump -U urbanmove -d urbanmove > /tmp
  ↳ /urbanmove_backup.sql

# Restore into separate database
sudo docker exec -i urbanmove-postgres psql -U urbanmove -d postgres -c "
  ↳ DROP DATABASE IF EXISTS urbanmove_restore;"
sudo docker exec -i urbanmove-postgres psql -U urbanmove -d postgres -c "
  ↳ CREATE DATABASE urbanmove_restore;"
cat /tmp/urbanmove_backup.sql | sudo docker exec -i urbanmove-postgres psql
  ↳ -U urbanmove -d urbanmove_restore

# Verify restored schema
sudo docker exec -i urbanmove-postgres psql -U urbanmove -d
  ↳ urbanmove_restore -c "\dt"
```

```
NOTICE: database "urbanmove_restore" does not exist, skipping
DROP DATABASE
CREATE DATABASE
SET
SET
CREATE TABLE
CREATE TABLE
CREATE INDEX
...

          List of relations
 Schema |          Name          | Type  | Owner
-----+-----+-----+-----
 public | congestion_events     | table | urbanmove
```

```

public | routes_cache      | table | urbanmove
public | routing_metrics       | table | urbanmove
(3 rows)

```

Exercise 7. Evidence Mapping

Question 1. Requirement to evidence map

Requirement	Evidence
Architecture design	architecture/exports/architecture.design.png
Budget control	evidence/screenshots/01-budget/budget.png
Private VPC + NAT + private subnets	vpc_with_subnets_and_routes.png, nat_gateway.png
Identity and access setup	iam_users.png
Edge protection and API entry	cloudfront.png, waf.png, api_gateway_routes.png
Service deployment + API validation	smoke_test_success.png
Observability + pipelines + DR evidence	cloudwatch_dashboard.png, pipelines_passed.png, backup/restore command output in Section 6

Exercise 8. Conclusion

This report records:

- implemented architecture,
- exact infrastructure and deployment commands,
- screenshot evidence currently available in repository,
- backup/restore drill commands and sample terminal output.